Resumen: Organización del Computador

1. Sistemas de Numeración y Aritmética

1.1. Conversión Hexadecimal - Binario

El sistema hexadecimal (base 16) es una forma compacta de representar binarios. Cada dígito hexadecimal corresponde exactamente a 4 bits.

- **Práctica:** Para convertir, agrupamos de a 4 bits de derecha a izquierda.
- **Ejemplo:** $(1011\ 0110)_2 \rightarrow B = 1011, 6 = 0110 \rightarrow (B6)_{16}$.

1.2. Punto Flotante (Estándar IEEE 754)

Representa números reales mediante la fórmula: $V = (-1)^S \times (1 + F) \times 2^{(E-127)}$.

Metodología: De Decimal a IEEE 754

1. **Signo (S):** 0 si es positivo, 1 si es negativo.
2. **Binario:** Convertir el número a binario (parte entera y fraccionaria).
3. **Normalizar:** Mover la coma para que quede $1, xxxxx$. La cantidad de lugares movidos es el **exponente real** (exp).
4. **Exponente de campo (E):** Calcular $E = exp + 127$. Convertir a 8 bits.
5. **Mantisa (F):** Tomar los 23 bits a la derecha de la coma.

2. Álgebra de Boole y Funciones

2.1. Minitérminos y Maxitérminos

- **Minitérminos (m_i):** Producto (AND) de todas las variables. La variable es directa si vale 1 y negada si vale 0. Se usan para la **Suma de Productos** (cuando la función $F = 1$).
- **Maxitérminos (M_i):** Suma (OR) de todas las variables. La variable es directa si vale 0 y negada si vale 1. Se usan para el **Producto de Sumas** (cuando $F = 0$).

2.2. Leyes de De Morgan

Cruciales para simplificar y convertir compuertas:

1. $(A \cdot B)' = A' + B'$
2. $(A + B)' = A' \cdot B'$

3. Lógica Combinacional y Simplificación

3.1. Mapas de Karnaugh (K-Maps)

Es un método gráfico para simplificar funciones booleanas. Permite identificar términos comunes visualmente para obtener la expresión mínima.

- **Celdas Adyacentes:** Solo pueden agruparse celdas en potencias de 2 $(1, 2, 4, 8 \dots)$.
- **Adyacencia de Bordes:** Las celdas de los extremos izquierdo/derecho y superior/inferior se consideran adyacentes (el mapa es un toroide).
- **Condiciones No Importa (Don't Care - X):** Se pueden usar como 1 si ayudan a agrandar un grupo, o como 0 si no son necesarias.

3.2. Módulos Combinacionales MSI

3.2.1. Decodificadores (Decoders)

Un decodificador de n entradas a 2^n salidas activa la salida correspondiente al número binario de entrada.

- **Práctica:** Son útiles para implementar funciones booleanas. Una función se puede realizar sumando (OR) las salidas del decodificador que corresponden a los minitérminos de dicha función.

3.2.2. Multiplexores (MUX)

Selecciona una de varias entradas de datos y la dirige a una sola salida mediante líneas de selección.

- **Regla de diseño:** Un MUX de 2^n entradas a 1 salida puede implementar cualquier función de $n + 1$ variables, conectando n variables a las entradas de selección y la variable restante (o su complemento/0/1) a las entradas de datos.

4. Aritmética del Computador

4.1. Sumadores

- **Semisumador (Half-Adder):** Suma dos bits (x, y).

$$S = x \oplus y \quad ; \quad C = x \cdot y$$

- **Sumador Completo (Full-Adder):** Suma dos bits y un acarreo de entrada (C_{in}).

$$S = x \oplus y \oplus C_{in} \quad ; \quad C_{out} = xy + C_{in}(x \oplus y)$$

4.2. Acarreo Anticipado (Look-Ahead Carry)

Para evitar el retardo del .ªcarreo en serie"(Ripple Carry), se calculan los acarreos en paralelo usando las funciones:

- **Generación (G_i):** $G_i = A_i \cdot B_i$ (se genera un acarreo).
- **Propagación (P_i):** $P_i = A_i \oplus B_i$ (el acarreo de entrada se propaga a la salida).

Fórmula clave: $C_{i+1} = G_i + P_i C_i$.

Tip de Examen: Sumador/Restador

Para convertir un sumador en restador de $A - B$, se utiliza el formato de Complemento a 2: 1. Se invierten los bits de B (usando compuertas XOR con una señal de control). 2. Se ingresa un 1 en el acarreo inicial (C_0).

5. Lógica Secuencial

A diferencia de la combinacional, en la lógica secuencial la salida depende de las entradas actuales y del **estado anterior** (memoria).

5.1. Elementos de Memoria: Latches y Flip-Flops

- **Latch SR:** Elemento básico de memoria. Tiene el problema de que la entrada $S = 1, R = 1$ es indeterminada.
- **Flip-Flop D (Data):** Captura el valor de la entrada D en el flanco de subida del reloj (CLK). Es el más usado para registros. Su ecuación característica es: $Q(t+1) = D$.
- **Flip-Flop JK:** Evita el estado indeterminado. Si $J = 1, K = 1$, la salida conmuta (toggle). Ecuación: $Q(t+1) = JQ' + K'Q$.
- **Flip-Flop T (Toggle):** Si $T = 1$, la salida cambia; si $T = 0$, se mantiene. Ecuación: $Q(t+1) = T \oplus Q$.

5.2. Tablas de Excitación

Son esenciales para el **diseño**. Responden a: *¿Qué entradas necesito para pasar del estado actual $Q(t)$ al estado siguiente $Q(t+1)$?*

$Q(t)$	$Q(t+1)$	**D**	**J**	**K**	**T**
0	0	0	0	X	0
0	1	1	1	X	1
1	0	0	X	1	1
1	1	1	X	0	0

Cuadro 1: Tablas de excitación para diseño de circuitos.

6. Análisis y Diseño de Circuitos Secuenciales

Metodología de Diseño (Máquinas de Estado)

1. **Diagrama de Estados:** Dibujar los estados y las transiciones según el enunciado.
2. **Tabla de Estados:** Listar Estado Actual, Entrada, Estado Siguiente y Salida.
3. **Asignación de Binario:** Asignar un código binario a cada estado (ej: $S_0 = 00, S_1 = 01 \dots$).
4. **Ecuaciones de Excitación:** Usar la tabla de excitación del Flip-Flop elegido y simplificar mediante Mapas de Karnaugh para hallar las funciones de las entradas (ej: D_A, J_B, K_B).
5. **Circuito:** Dibujar el esquema lógico.

6.1. Modelos de Mealy y Moore

- **Moore:** La salida depende **solo** del estado actual. (La salida se anota dentro del círculo del estado).
- **Mealy:** La salida depende del estado actual **y** de la entrada actual. (La salida se anota en la flecha de transición).

7. Mapeo de Memoria

El objetivo es organizar diferentes chips de memoria (RAM/ROM) en el espacio de direcciones del procesador utilizando un decodificador para activar el chip correcto (señal Chip Select - CS).

7.1. Conceptos Fundamentales

- **Bus de Direcciones (A_n):** Determina la capacidad máxima de direccionamiento (2^{bits}).
- **Líneas de Dirección del Chip:** Si un chip es de 2^k palabras, utiliza las líneas desde A_0 hasta A_{k-1} para direccionar internamente sus celdas.
- **Líneas de Selección (Decodificación):** Las líneas de direcciones más significativas ($A_{max} \dots A_k$) se conectan a un decodificador para generar los **Chip Select (CS)**.

7.2. Cálculo de Capacidad y Rangos

Para un bloque de memoria:

1. **Tamaño del bloque:** Si el chip tiene k líneas de dirección, su tamaño es 2^k. Ej: 10 bits $(A_0 - A_9) = 2^{10} = 1024$ posiciones (1K).
2. **Dirección de Inicio:** Se obtiene poniendo en 0 todas las líneas de dirección internas del chip ($A_{k-1} \dots A_0$) y manteniendo fijas las líneas de selección.
3. **Dirección Final:** Se obtiene poniendo en 1 todas las líneas de dirección internas del chip.

Posiciones Imagen (Espejo)

Ocurren cuando algunas líneas del bus de direcciones **no se conectan** ni al chip ni al decodificador.

- **Efecto:** El mismo dato aparece en múltiples direcciones diferentes.
- **Cómo detectarlas:** Si hay n líneas "sueltas", habrá 2^n imágenes del mismo bloque de memoria.

8. Registros y Contadores

8.1. Registros de Desplazamiento

Formados por una cadena de Flip-Flops D donde la salida de uno es la entrada del siguiente.

- **Entrada/Salida Serie:** Los datos entran bit a bit.
- **Entrada/Salida Paralelo:** Los datos se cargan o leen todos al mismo tiempo (requiere más pines).

8.2. Contadores Sincrónicos

Se diseñan como una Máquina de Estados donde el estado siguiente es simplemente el valor actual + 1.

- **Diseño:** Se definen los estados (ej. $00 \rightarrow 01 \rightarrow 10 \rightarrow 11 \rightarrow 00$), se busca la tabla de excitación y se simplifica para hallar las entradas de los Flip-Flops.

9. Arquitectura del Procesador (Datapath y Control)

El procesador se divide principalmente en dos unidades: la de **Datos** (Datapath) y la de **Control**.

9.1. El Camino de Datos (Datapath)

Es el conjunto de elementos que operan sobre los datos (ALU, Registros, PC). El ciclo de ejecución típico sigue estos pasos:

1. **Fetch (Búsqueda):** Se lee la instrucción de la memoria apuntada por el *Program Counter* (PC).
2. **Decode (Decodificación):** La Unidad de Control identifica qué operación debe realizarse.
3. **Execute (Ejecución):** La ALU realiza la operación (suma, resta, etc.) o se accede a memoria.
4. **Write Back (Escritura):** El resultado se guarda en el registro de destino.

9.2. Unidad de Control

Es el cerebro del procesador. Genera señales (como *MemRead*, *RegWrite*, *ALUSel*) que activan o desactivan componentes del Datapath según la instrucción actual.

10. Jerarquía de Memoria y Caché

Para solucionar la brecha de velocidad entre el procesador (muy rápido) y la memoria principal (lenta), se utilizan memorias intermedias llamadas **Caché**.

10.1. Principios de Localidad

- **Localidad Temporal:** Si se accede a un dato, es probable que se vuelva a acceder pronto (ej. un bucle).
- **Localidad Espacial:** Si se accede a un dato, es probable que se acceda pronto a los datos vecinos (ej. un arreglo).

Tip de Examen: Parámetros de Caché

- **Hit (Acierto):** El dato buscado está en el nivel superior de memoria.
- **Miss (Fallo):** El dato no está; se debe buscar en un nivel inferior, lo que genera una *penalización por fallo* (tiempo extra).
- **Bloque (Línea):** La unidad mínima de información que se transfiere entre niveles.

11. Conclusión del Resumen

Este documento cubre desde la representación básica de datos hasta la integración de sistemas complejos de memoria y procesamiento. Para el examen, se recomienda practicar especialmente el **Mapeo de Memoria** y el **Diseño de FSM (Máquinas de Estado)**, ya que combinan toda la teoría anterior.